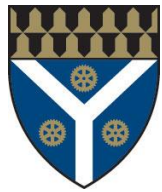




Scaling Virtual Worlds: A Distributed Server Architecture for Real-Time Multiplayer Games

June Yoo, Dr. Mike Shah, Department of Computer Science, Yale University



OVERVIEW

Modern online games have been designed exclusively around a single-server deployment paradigm. However, recent market developments in gaming and virtual reality suggest the need for a more scalable approach to real-time virtual experiences.

In this project, we design and implement a distributed networking library for real-time games. The underlying architecture divides the state and execution of a game between an arbitrary number of participating nodes.

Upon testing various deployment configurations of a model game, we find that our design increases total bandwidth as desired, showing strong potential for working at scale.

KEY FINDINGS

- > Games have application-specific features which allow us to make specific design choices for distributed systems: this includes strong **spatial locality** and tolerance to **transient inconsistency**
- > The largest commercial real-time games currently on the market generally support up to 100 concurrent players per match.
- > We see an opportunity to distribute game servers by **splitting game state and execution** between nodes by game object.
- > With our new architecture, we have verified that we can host significantly more players than a single-server design, without impacting other areas of performance.

BACKGROUND

Games are essentially state machines which may be represented as a collection of *game objects*. Objects change at some tickrate according to an *update function* acting on their own state and the states of other objects in their vicinity (which we call the *area-of-interest*).

The largest real-time commercial games (*Fortnite*, *PUBG*, *CoD*) support up to ~100 players per match, with a standard server tick rate between 20-30 Hz. *EVE Online* is frequently cited as having the highest-ever concurrent player count on one server at 2670, but with a tick rate of 1 Hz, which is arguably not real-time. There are virtually no real-time titles which exceed player capacities of 200-300.

ARCHITECTURE

Our server architecture is largely based off the *Colyseus* protocol. In our implementation, this essentially consists of two components per node: the replica manager and object locator.

The **replica manager** manages the game state. In particular, each game object has a single *primary copy*, and all other nodes may or may not have a *replica copy* of the object. The node is only concerned about replica objects whose state may be relevant to updating its primary objects; the mechanism for this is described below. The replica manager directly corresponds with the owner of any relevant primary objects to receive updates to its replicas. At set intervals, the replica manager checks each of its replicas to verify that they are still within its primaries' areas-of-interest, and discards replicas for which the condition is not met.

The **object locator** enables discovery of relevant game state: primary objects publish their areas-of-interest to a distributed range-query hash table, and matches are reported back to the replica manager. This area-of-interest calculation leverages a strong spatial locality assumption that is true in most games: we can simply query a bounding box around each object to search for relevant state. We optimize this by predictively adjusting the area-of-interest according to the objects' current velocities.

Diverging from the original design, we transition the core game logic to an event-driven model, which allows us to derive strong guarantees about consistency since state is determined per-object (proof omitted here). We also notice that this model also naturally results in graceful degradation upon node failures,

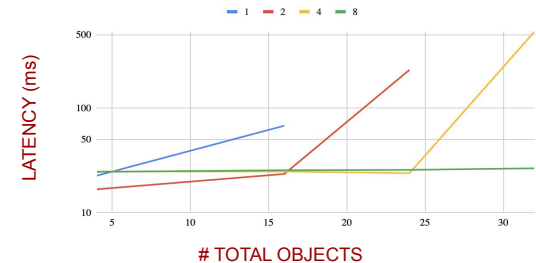
EXPERIMENTS

We test the effectiveness of our design by running various deployment configurations of a simple model game, consisting of players randomly walking around on a limited 2D grid.

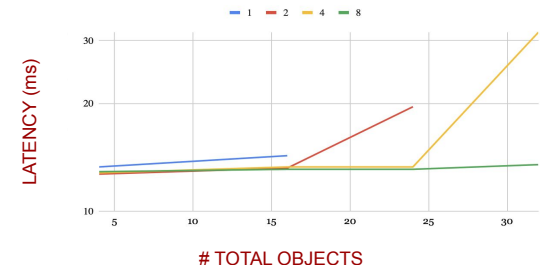
For each trial, we fix the number of servers and scale the total number of objects (the majority of which are "player" avatars, each inputting commands at 2 Hz) measure the average and tail latencies over a window.

We introduce a base artificial latency to each server of 10m, as well as artificial bandwidth, simulating real network conditions.

MEAN LATENCY BY CONFIGURATION



TAIL (p99) LATENCY BY CONFIG.



RESULTS

The results broadly align with our expectations, showing a **positive correlation between overall player capacity and the number of nodes**. Specifically, the data seems to indicate that all configurations perform relatively similarly in sparsely populated scenarios but have different points of failure. Congestion collapse occurs after the 16-player case for the single-server configuration, but distributing the server to two nodes increases this to at least 24, and distributing to four increases it to at least 32. Hence, our architecture seems to successfully accomplish the goal of scaling a single virtual world well beyond the limitations of a single machine.

Importantly, there is also little to no variability in a single configuration's performance before its eventual collapse. For instance, the eight-node configuration experiences virtually **no slowdown** as its player population increases from 4 to 32.